P1132R1
out_ptr - a scalable output pointer abstraction

Published Proposal, 2018-08-11

Authors:

JeanHeyd Meneide

Todor Buyukliev

Isabella Muerte

Audience:

LEWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/ThePhD/future_cxx/blob/master/papers/source/d1132.bs

Current:

https://rawgit.com/ThePhD/future_cxx/master/papers/d1132.html

Implementation:

https://github.com/ThePhD/out_ptr

Reply To:

JeanHeyd Meneide | @thephantomderp

Abstract

out_ptr is an abstraction to bring both C APIs and smart pointers back into the promised land by creating a temporary pointer-to-pointer that updates the smart pointer when it destructs.

Table of Contents

1 Revision History

1.1 Revision 0
1.2 Revision 1

2 Motivation

3 Design Considerations
3.1 Synopsis
3.2 Header and Feature Macro
3.3 Overview
3.4 Safety
3.5 Casting Support
3.6 Reallocation Support

4 Implementation Experience
4.1 Why Not Wrap It?

5 Performance
5.1 For `std::out_ptr`
5.2 For `std::inout_ptr`

6 Bikeshed
6.1 Alternative Specification
6.2 Naming

7 Proposed Changes
7.1 Proposed Feature Test Macro and Header
7.2 Intent
7.3 Proposed Wording

8 Acknowledgements

References
Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release.

§ 1.2. Revision 1

Add wording. Incorporate wording feedback. Eliminate CTAD design. Add a few more words about implementation experience.

§ 2. Motivation

We have very good tools for handling unique and shared resource semantics, alongside more coming with Intrusive Smart Pointers. Independently between several different companies, studios, and shops -- from VMWare and Microsoft to small game development startups -- a common type has been implemented. It has many names: `ptrptr`, `OutPtr`, `PtrToPtr`, `out_ptr`, WRL::ComPtrRef and even unary operator& on CComPtr. It is universally focused on one task: making it so a smart pointer can be passed as a parameter to a function which uses an output pointer parameter in C API functions (e.g., `my_type**`).

This paper is a culmination of a private survey of types from the industry to propose a common, future-proof, high-performance `out_ptr` type that is easy to use. It makes interop with pointer types a little bit simpler and easier for everyone who has ever wanted something like `my_c_function( &my_unique );` to behave properly.

In short: it's a thing convertible to a `T**` that updates the smart pointer it is created with when it goes out of scope.

§ 3. Design Considerations

The core of `out_ptr`'s (and `inout_ptr`'s) design revolves around avoiding the mistakes of the past, preventing continual modification of new smart pointers and outside smart pointers's interfaces to perform the same task, and enabling some degree of performance efficiency without having to wrap every C API function.

§ 3.1. Synopsis

The function template's full specification is:

```
namespace std {
        template <class Pointer, class Smart, class... Args>
        auto out_ptr(Smart& s, Args&&... args) noexcept
        -> out_ptr_t<Smart, Pointer, Args...>;

        template <class Smart, class... Args>
        auto out_ptr(Smart& s, Args&&... args) noexcept
        -> decltype(out_ptr<PointerOf<Smart>>(s, std::forward<Args>(args)..

        template <class Pointer, class Smart, class... Args>
        auto inout_ptr(Smart& s, Args&&... args) noexcept
        -> inout_ptr_t<Smart, Pointer, Args...>;

        template <class Smart, class... Args>
        auto inout_ptr(Smart& s, Args&&... args) noexcept
        -> decltype(inout_ptr<PointerOf<Smart>>(s, std::forward<Args>(args)
}
```

Where `PointerOf` is the `::pointer` type, then `::element_type*`, then `class std::pointer_traits<Smart>::element_type*` in that order. The return type `out_ptr_t` and its sister type `inout_ptr_t` are templated types and must at-minimum have the following:

```
template <class Smart, class Pointer, class... Args>
struct out_ptr_t {
        out_ptr_t(Smart&, Args...);
        ~out_ptr_t () noexcept;
        operator Pointer* () noexcept;
        operator void** () noexcept;
};

template <class Smart, class Pointer, class... Args>
struct inout_ptr_t {
        inout_ptr_t(Smart&, Args...);
        ~inout_ptr_t () noexcept;
        operator Pointer* () noexcept;
        operator void** () noexcept;
};
```

We specify "at minimum" because we expect users to override this type for their own shared, unique, handle-alike, reference-counting, and etc. smart pointers. The destructor of `~out_ptr_t()` calls `.reset()` on the stored smart pointer of type `Smart` with the stored pointer of type `Pointer` and arguments stored as `Args...`. `~inout_ptr_t()` does the same, but with the additional caveat that the constructor for `inout_ptr_t(Smart&, Args&&...)` also calls `.release()`, so that a `reset` doesn't double-delete a pointer that the expected re-allocating API used with `inout_ptr` already handles.

§ 3.2. Header and Feature Macro

The target header is `<memory>`. The desired feature is `__cpp_lib_out_ptr`. See §7.1 Proposed Feature Test Macro and Header for further discussion about other potential targets.

§ 3.3. Overview

`out_ptr`/`inout_ptr` are free functions meant to be used for C APIs:

```
error_num c_api_create_handle(int seed_value, int** p_handle);
error_num c_api_re_create_handle(int seed_value, int** p_handle);
void c_api_delete_handle(int* handle);

struct resource_deleter {
        void operator()( int* handle ) {
                c_api_delete_handle(handle);
        }
};
```

Given a smart pointer, it can be used like so:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_create_handle(
        24, std::out_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from the C API function
```

Or, in the re-create (reallocation) case:

```
std::unique_ptr<int, resource_deleter> resource(nullptr);
error_num err = c_api_re_create_handle(
        24, std::inout_ptr(resource)
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from the C API function
```

§ 3.4. Safety

This implementation uses a pack of `...Args` in the signature of `out_ptr` to allow it to be used with other types whose `.reset()` functions may require more than just the pointer value to form a valid and proper smart pointer. This is the case with `std::shared_ptr` and `boost::shared_ptr`:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
        24, std::out_ptr(resource, resource_deleter{})
);
if (err == C_API_ERROR_CONDITION) {
        // handle errors
}
// resource.get() the out-value from
// the C API function
```

Additional arguments past the smart pointer stored in `out_ptr`'s implementation-defined return type will perfectly forward these to whatever `.reset()` or equivalent implementation requires them. If the underlying pointer does not require such things, it may be ignored or discarded (optionally, with a compiler error using a static assert that the argument will be ignored for the given type of smart pointer).

Of importance here is to note that `std::shared_ptr` can and will overwrite any custom deleter present when called with just `.reset(some_pointer);`. Therefore, we make it a compiler error to not pass in a second argument when using `std::shared_ptr` without a deleter:

```
std::shared_ptr<int> resource(nullptr);
error_num err = c_api_create_handle(
        42, std::out_ptr(resource)
); // ERROR: deleter was changed
   // to an equivalent of
   // std::default_delete!
```

It is likely the intent of the programmer to also pass the fictional `c_api_delete_handle` function to this: the above constraint allows us to avoid such programmer mistakes.

§ 3.5. Casting Support

There are also many APIs (COM-style APIs, base-class handle APIs, type-erasure APIs) where the initialization requires that the type passed to the function is of some fundamental (`void**`) or base type that does not reflect what is stored exactly in the pointer. Therefore, it is necessary to sometimes specify what the underlying type `out_ptr` uses is stored as.

It is also important to note that going in the *opposite* direction is also highly desirable, especially in the case of doing API-hiding behind an e.g. `void*` implementation. `out_ptr` supports both scenarios

with an optional template argument to the function call.

For example, consider this DirectX Graphics Infrastructure Interface (DXGI) function on `IDXGIFactory6`:

```
HRESULT EnumAdapterByGpuPreference(
        UINT Adapter,
        DXGI_GPU_PREFERENCE GpuPreference,
        REFIID riid,
        void** ppvAdapter
);
```

Using `out_ptr`, it becomes trivial to interface with it using an exemplary `std::unique_ptr<IDXGIAdapter, ComDeleter> adapter`:

```
HRESULT result = dxgi_factory.
EnumAdapterByGpuPreference(0,
        DXGI_GPU_PREFERENCE_MINIMUM_POWER,
        IID_IDXGIAdapter,
        std::out_ptr<void*>(adapter)
);
if (FAILED(result)) {
        // handle errors
}
// adapter.get() contains strongly-typed pointer
```

No manual casting, `.release()` fiddling, or `.reset()` is required: the returned type from `out_ptr` handles that.

§ 3.6. Reallocation Support

In some cases, a function given a valid handle/pointer will delete that pointer on your behalf before performing an allocation in the same pointer. In these cases, just `.reset()` is entirely redundant and dangerous because it will delete a pointer that it does not own. Therefore, there is a second abstraction called `inout_ptr`, so aptly named because it is both an input (to be deleted) and an output (to be allocated post-delete). `inout_ptr`'s semantics are exactly like `out_ptr`'s, just with the additional requirement that it calls `.release()` on the smart pointer upon constructing the temporary `inout_ptr_t`.

This can be heavily optimized in the case of `unique_ptr`, but to do so from the outside requires Undefined Behavior or modification of the standard library. See §5.2 For std::inout_ptr for further explication.

§ 4. Implementation Experience

This library has been brewed at many companies in their private implementations, and implementations in the wild are scattered throughout code bases with no unifying type. As noted in §2 Motivation, Microsoft has implemented this in `WRL::ComPtrRef`. Its earlier iteration -- `CComPtr` -- simply overrode `operator&`. We assume they prefer the former after having forced the need with `CComPtr` for `std::addressof`. VMWare has a type that much more closely matches the specification in this paper, titled `Vtl::OutPtr`. The primary author of this paper wrote and used `out_ptr` for over 5 years in their code base working primarily with graphics APIs such as DirectX and OpenGL, and more recently Vulkan. They have also seen a similar abstraction in the places they have interned at.

The primary author of [p0468] in pre-r0 days also implemented an overloaded `operator&` to handle interfacing with C APIs, but was quickly talked out of actually proposing it when doing the proposal. That author has joined in on this paper to continue to voice the need to make it easier to work with C APIs without having to wrap the function.

Given that many companies, studios and individuals have all invented the same type independently of one another, we believe this is a strong indicator of agreement on an existing practice that should see a proposal to the standard.

A full implementation with UB and friendly optimizations is available in the repository. The type has been privately used in many projects over the last four years, and this public implementation is already seeing use at companies today. It has been particularly helpful with many COM APIs, and the re-allocation support in `inout_ptr` has been useful for FFMPEG's functions which feature reallocation support in their functions (e.g., `avformat_open_input`).

§ 4.1. Why Not Wrap It?

A common point raised while using this abstraction is to simply "wrap the target function". We believe this to be a non-starter in many cases: there are thousands of C API functions and even the most dedicated of tools have trouble producing lean wrappers around them. This tends to work for one-off functions, but suffers scalability problems very quickly.

Templated intermediate wrapper functions which take a function, perfectly forwards arguments, and attempts to generate e.g. a `unique_ptr` for the first argument and contain the boiler plate within itself also causes problems. Asides from the (perhaps minor) concern that such a wrapping function disrupts any auto-completion or tooling, the issue arises that C libraries -- even within themselves -- do not agree on where to place the `some_c_type**` parameter and detecting it properly to write a generic function to automagically do it is hard. Even within the C standard library, some functions have output parameters in the beginning and others have it at the end. The disparity grows when users pick up libraries outside the standard.

§ 5. Performance

Many C programmers in our various engineering shops and companies have taken note that manually re-initializing a `unique_ptr` when internally the pointer value is already present has a measurable performance impact.

Teams eager to squeeze out performance realize they can only do this by relying on type-punning shenanigans to extract the actual value out of `unique_ptr`: this is expressly undefined behavior. However, if an implementation of `out_ptr` could be friended or shipped by the standard library, it can be implemented without performance penalty.

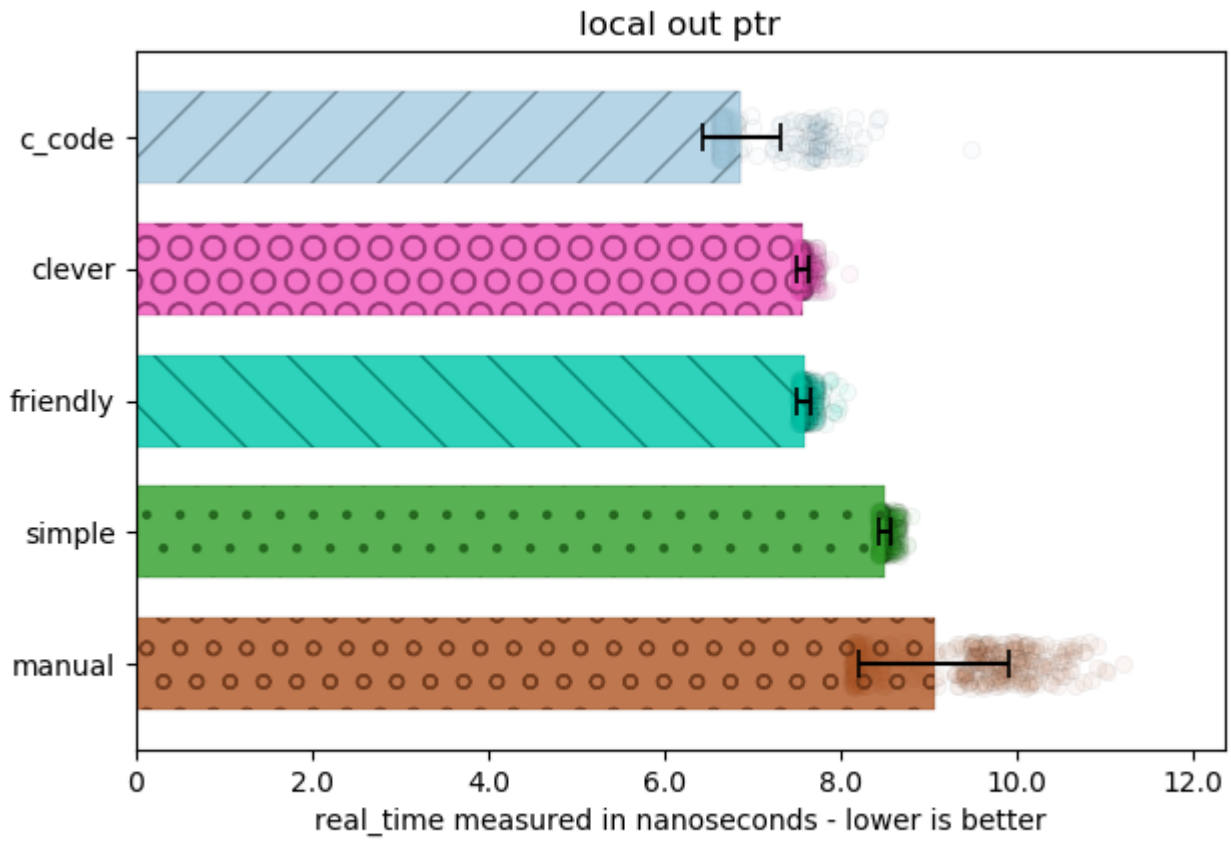
Below are some graphs indicating the performance metrics of the code. 5 categories were measured:

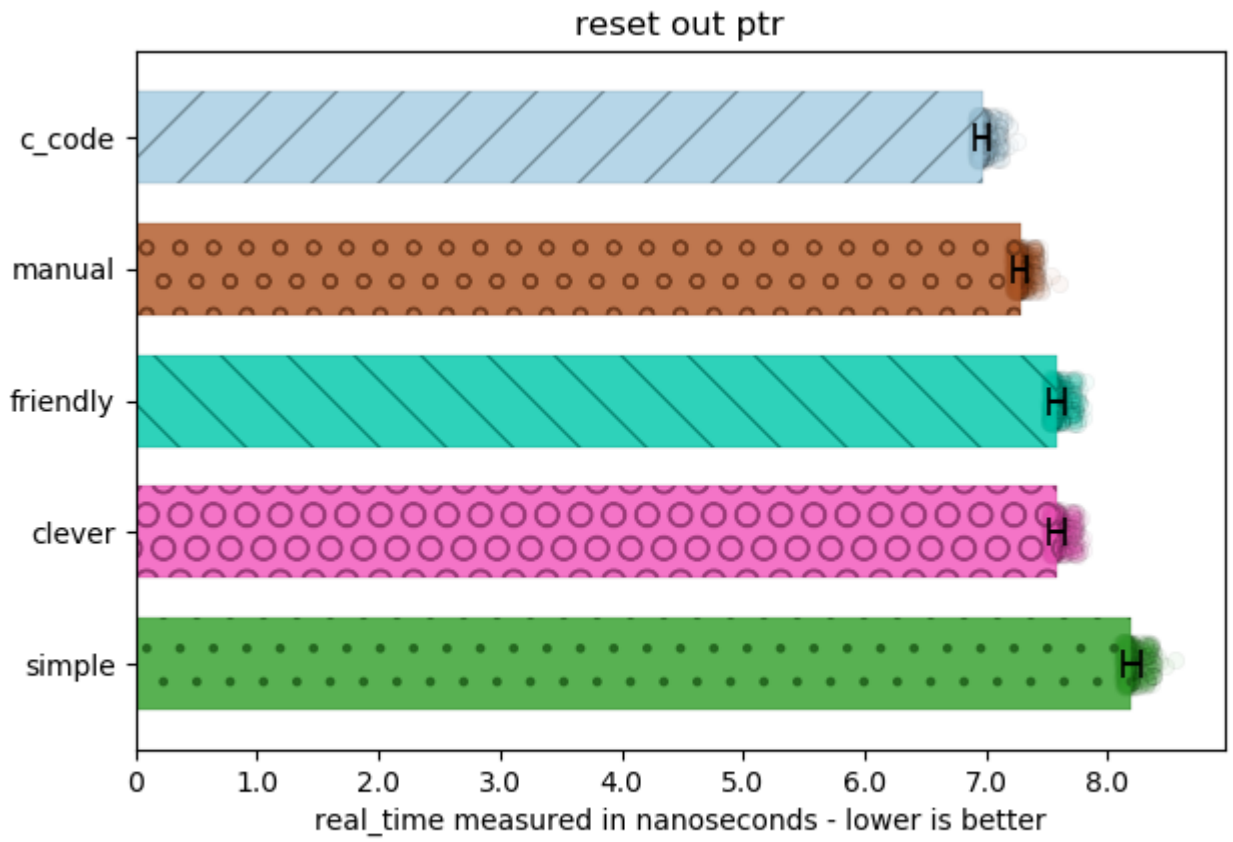
- "c_code": handwritten C code, which does not use this idiom
- "clever": uses UB to alias the pointer value stored in `std::unique_ptr`
- "friendly": modifies VC++'s, libc++'s, and libstdc++'s `std::unique_ptr`s to allow the implementation to friend the `out_ptr` implementation, to access the internals without UB
- "manual": does the work by-hand using reset/release from a `std::unique_ptr`
- "simple": a `out_ptr` implementation that naively resets

The full JSON data for these benchmarks is available in the repository, as well as all of the code necessary to run the benchmarks across all platforms with a simple CMake build system.

§ 5.1. For `std::out_ptr`

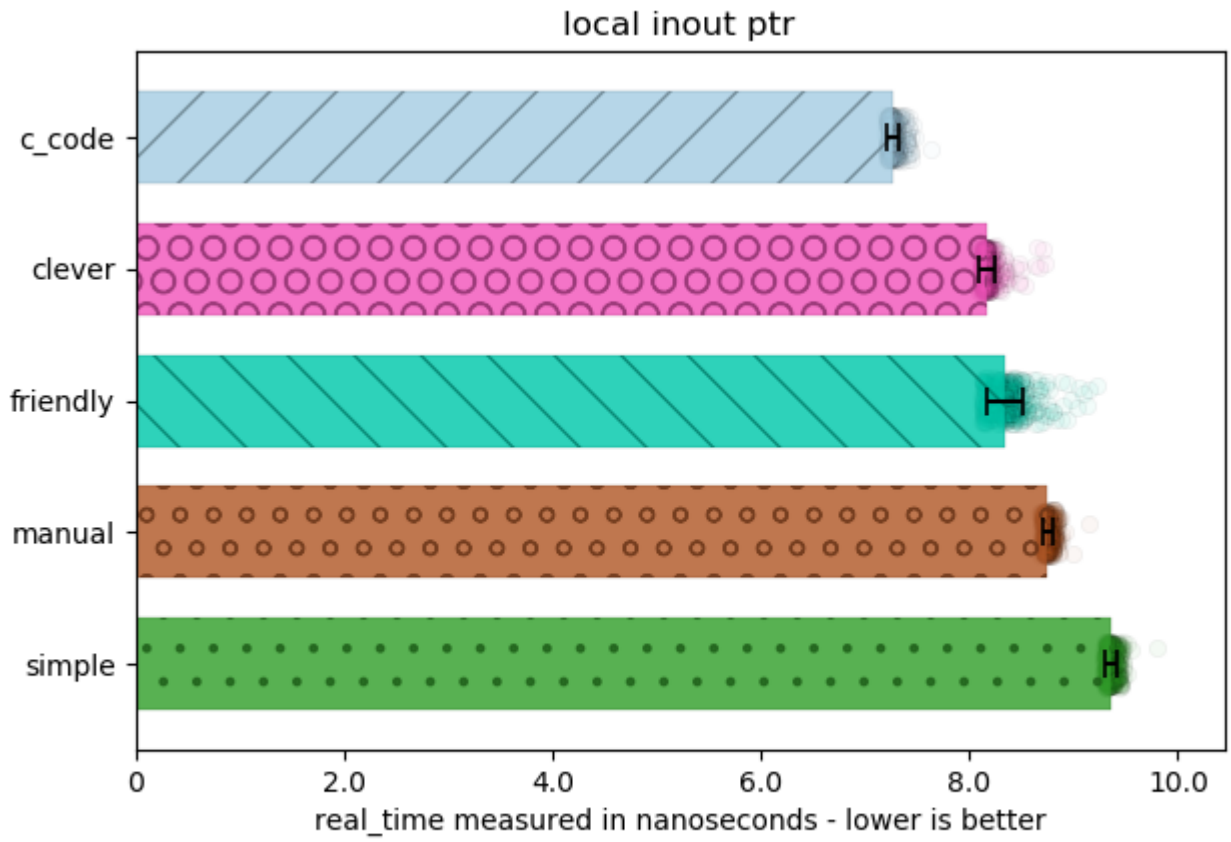
You can observe two graphs for two common `unique_ptr` usage scenarios, which are using the pointer locally and discarding it ("local"), and resetting a pre-existing pointer ("reset") for just an output pointer:

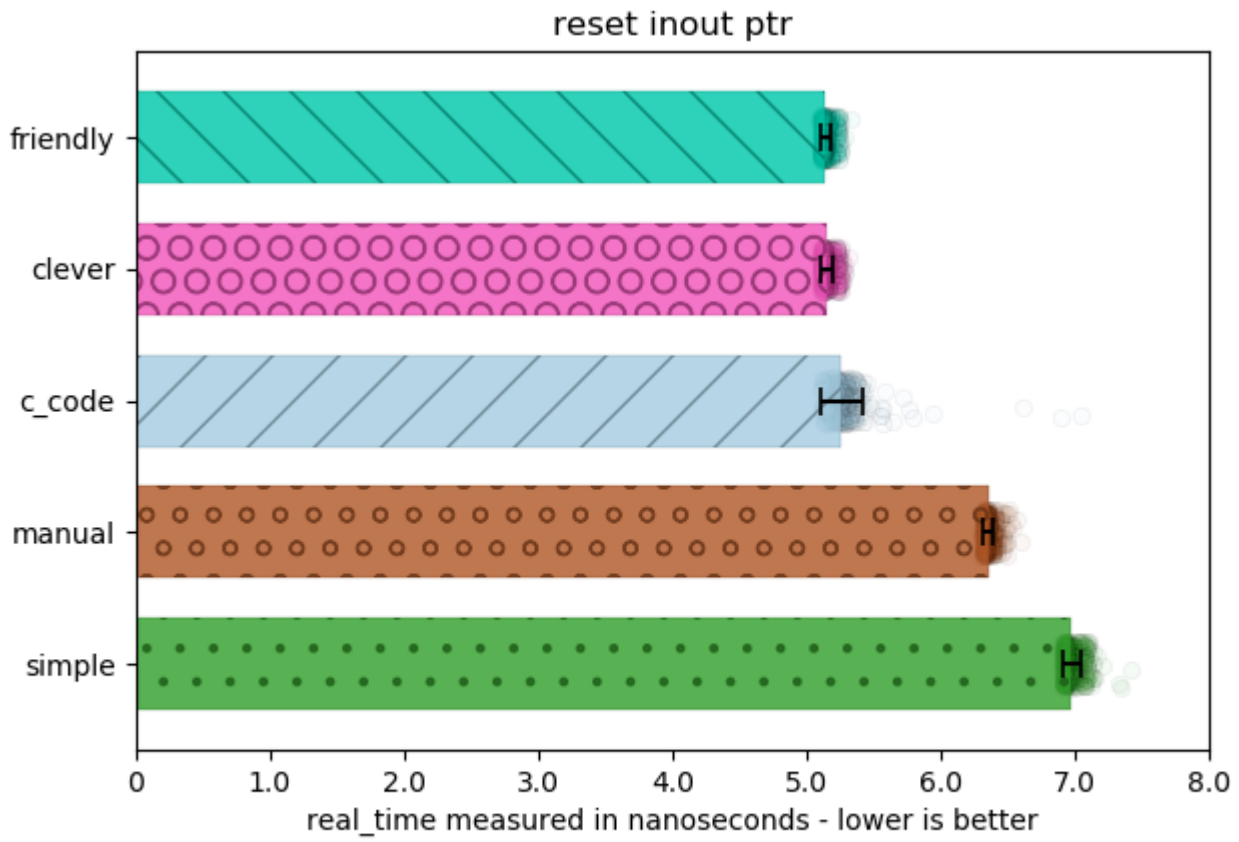

local out ptr
c_code
clever
friendly
simple
manual
0
2.0
4.0
6.0
8.0
10.0
12.0
real_time measured in nanoseconds - lower is better



§ 5.2. For `std::inout_ptr`

The speed increase here is even more dramatic: reseating the pointer through `.release()` and `.reset()` is much more expensive than simply aliasing a `std::unique_ptr` directly. Places such as VMWare have to perform Undefined Behavior to get this level of performance with `inout_ptr`: it would be much more prudent to allow both standard library vendors and users to be able to achieve this performance without hacks, tricks, and other I-promise-it-works-I-swear pledges.


local inout ptr
c_code
clever
friendly
manual
simple
0
2.0
4.0
6.0
8.0
10.0
real_time measured in nanoseconds - lower is better



§ 6. Bikeshed

As with every proposal, naming, conventions and other tidbits not related to implementation are important. This section is for pinning down all the little details to make it suitable for the standard.

§ 6.1. Alternative Specification

The authors of this proposal know of two ways to specify this proposal's goals.

The first way is to specify both functions `out_ptr` and `inout_ptr` as factories, and then have their types named differently, such as `out_ptr_t` and `inout_ptr_t`. The factory functions and their implementation will be fixed in place, and users would be able to (partially) specialize and customize `std::out_ptr_t` and `std::inout_ptr_t` for types external to the stdlib for maximum performance tweaking and interop with types like `boost::shared_ptr`, `my_lib::local_shared_ptr`, and others. This is the direction this proposal takes.

The second way is to specify the class names to be `std::out_ptr` / `std::inout_ptr`, and then used Template Argument Deduction for Class Templates from C++17 to give a function-like appearance to their usage. Users can still specialize for types external to the standard library. This approach is more Modern C++-like, but contains a caveat.

Part of this specification is that you can specify the stored pointer for the underlying implementation of `out_ptr` as shown in §3.5 Casting Support. Template Argument Deduction for Class Templates does not allow partial specialization (and for good reason, see the interesting example of `std::tuple<int, int>{1, 2, 3}`). The "Deduction Guides" (or CTAD) approach would accommodate §3.5 Casting Support using functions with a more explicit names, such as `out_ptr_cast<void*>( ... );` and `inout_ptr_cast<void*>( ... );`.

The authors have settled on the approach in §3.1 Synopsis. We believe this is the most robust and easiest to use: singular names tend to be easier to teach and use for both programmers and tools.

§ 6.2. Naming

Naming is hard, and therefore we provide a few names to duke it out in the Bikeshed Arena:

For the `out_ptr` part:

- out_ptr
- c_ptr
- c_out_ptr
- out_c_ptr
- alloc_c_ptr
- out_smart
- ptrptr
- ptr_to_ptr
- ptr_to_smart
- ptr_ref

For the `inout_ptr` part:

- inout_ptr
- c_in_ptr

- c_inout_ptr
- inout_c_ptr
- realloc_c_ptr
- inout_smart,
- realloc_ptr_to_ptr
- realloc_ptr_to_smart
- realloc_ptr_ref

As a pairing, `out_ptr` and `inout_ptr` are the most cromulent and descriptive in the authors' opinions. The type names would follow suit as `out_ptr_t` and `inout_ptr_t`. However, there is an argument for having a name that more appropriately captures the purpose of these abstractions. Therefore, `c_out_ptr` and `c_inout_ptr` would be even better, and the shortest would be `c_ptr` and `c_in_ptr`.

§ 7. Proposed Changes

The following wording is for the Library section, relative to [n4762]. This feature will go in the `<memory>` header, and is added to §19.11 [**utilities.smartptr**], at the end as subsection 9.

§ 7.1. Proposed Feature Test Macro and Header

This should be available with the rest of the smart pointers, and thusly be included by simply including `<memory>`. If there is a desire for more fine-grained control, then we recommend the header `<out_ptr>` (subject to change based on bikeshed painting above). There has been some expressed desire for wanting to provide more fine-grained control of what entities the standard library produces when including headers: this paper does not explicitly propose adding such headers or doing such work, merely making a recommendation if this direction is desired by WG21.

The proposed feature test macro for this is `__cpp_lib_out_ptr`. The exposure of `__cpp_lib_out_ptr` denotes the existence of both `inout_ptr` and `out_ptr`, as well as its customization points `out_ptr_t` and `inout_ptr_t`.

§ 7.2. Intent

The intent of this wording is to allow implementers the freedom to implement the return type from `out_ptr` as they so choose, so long as the following criteria is met:

- the return type is of the name `inout_ptr_t`/`out_ptr_t` and is a template with 3 template parameters;
- the destructor of `inout_ptr_t`/`out_ptr_t` properly re-seats the pointer owned/stored by whatever smart/fancy pointer is passed as the first argument to `out_ptr`;
- the proper implicit conversion operators are added to the type,
- the standard library implementation itself does not specialize `inout_ptr_t`/`out_ptr_t`'s templates, either fully or partially, so that the user can override the behavior of these templates as they so choose;
- `std::shared_ptr` used with the `out_ptr` or `inout_ptr` functions will produce a diagnostic if it is called without a second argument meant to be passed as the deleter to a `.reset()` call; and
- `std::shared_ptr` used with `inout_ptr_t` will always result in a diagnostic because it is impossible to completely release the resource from its shared ownership.

The goals of the wording are to not restrict implementation strategies (e.g., a `friend` implementation as benchmarked above for `unique_ptr`, or maybe a UB/IB implementation as also documented above). It is also explicitly meant to error for smart pointers whose `.reset()` call may reset the stored deleter (á la `boost::shared_ptr`/`std::shared_ptr`) and to catch programmer errors.

§ 7.3. Proposed Wording

Modify §19.10.1 In general [**memory.general**] as follows:

[1] The header defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct multiple objects in uninitialized memory buffers (19.10.3–19.10.11). The header also defines the templates unique_ptr, shared_ptr, weak_ptr, out_ptr_t, inout_ptr_t, and various function templates that operate on objects of these types (19.11).

Add §19.10.2 Definitions [**memory.defns**] as follows:

[1] Definition: Let `POINTER_OF(T)` denote a type that is:

— the type of `T::pointer` if the qualified-id `T::pointer` is valid and denotes a type, or

— the type of `std::add_pointer_t<typename T::element_type>` if the qualified-id `T::element_type` is valid and denotes a type, or

— the type of `std::add_pointer_t<typename std::pointer_traits<T>::element_type>` if `std::pointer_traits<T>::element_type` is valid,

— otherwise, `decltype(addressof(*declval<T>()))`

Add to §19.10.3 (previously §19.10.2) Header `<memory>` synopsis [**memory.syn**] the `out_ptr`, `inout_ptr`, `out_ptr_t` and `inout_ptr_t` functions and types:

```
// 19.11.9, out_ptr_t
template <class Smart, class Pointer, class... Args>
  struct out_ptr_t;

// 19.11.10, out_ptr
template <class Pointer, class Smart, class... Args>
  auto out_ptr(Smart& s, Args&&... args) noexcept
  -> out_ptr_t<Smart, Pointer, Args...>;

template <class Smart, class... Args>
  auto out_ptr(Smart& s, Args&&... args) noexcept
  -> decltype(out_ptr<PointerOf<Smart>>(s, std::forward<Args>(args)...))

// 19.11.11, inout_ptr_t
template <class Smart, class Pointer, class... Args>
  struct inout_ptr_t;

// 19.11.12, inout_ptr
template <class Pointer, class Smart, class... Args>
  inout_ptr_t<Smart, Pointer, Args...> inout_ptr(Smart& s, Args&&... arg

template <class Smart, class... Args>
  auto inout_ptr(Smart& s, Args&&... args) noexcept
  -> decltype(inout_ptr<PointerOf<Smart>>(s, std::forward<Args>(args)...
```

Insert §19.11.9 [**out_ptr.class**]:

19.11.9 **Class Template** `out_ptr_t` [**out_ptr.class**]

[1] *out_ptr_t* is a type used with smart pointers (**19.11**) and types which are designed on the same principles to interoperate easily with functions that use output pointer parameters. [Note — For example, a function of the form `void foo(void**)` — end note].

[2] *out_ptr_t* may be specialized (**12.6.5**) for user-defined types and shall meet the observable behavior in the rest of this section.

```
namespace std {

  template <class Smart, class Pointer, class... Args>
  struct out_ptr_t {
    // 19.11.9.1, constructors
    out_ptr_t(Smart&, Args...) noexcept;
    out_ptr_t(out_ptr_t&&) noexcept;
    out_ptr_t(const out_ptr_t&) noexcept = delete;

    // 19.11.9.2, assignment
    out_ptr_t& operator=(out_ptr_t&&) noexcept;
    out_ptr_t& operator=(const out_ptr_t&) noexcept = delete;

    // 19.11.9.3, destructors
    ~out_ptr_t();

    // 19.11.9.4, conversion operators
    operator Pointer*() noexcept;  // if Pointer not void*
    operator void**() noexcept;

  private:
    Smart* s; // exposition only
    tuple<Args> a; // exposition only
    Pointer p; // exposition only
  };

}
```

[2] If `Smart` is a specialization of `shared_ptr` and `sizeof...(Args) == 0`, the program is ill-formed. `Pointer` shall meet the `Cpp17NullablePointer` requirements (**15.5.3.3**).

[3] [*Note:* It is typically a user error to reset a `shared_ptr` without specifying a deleter, as `std::shared_ptr` will replace a custom deleter with the default deleter upon usage of `.reset()`, as specified in 19.11.3.4. — *end Note*]

19.11.9.1 Constructors [**out_ptr.class.ctor**]

```
out_ptr_t(Smart& smart, Args... args) noexcept;
```

[1] Effects: constructs an object of `out_ptr_t` and stores the arguments to be used for destructor.

[2] Equivalent to:

```
out_ptr_t(Smart& smart, Args... args) noexcept :
  s(&smart),
  a(std::forward<Args>(args)...),
  p(static_cast<Pointer>(smart.get()))
{}.
```

```
out_ptr_t(out_ptr&& rhs) noexcept;
```

[3] Effects: moves all elements of `rhs` into `*this`, then sets `rhs.p` to `nullptr` so that the destructor's effects do not apply.

[4] Equivalent to:

```
out_ptr_t(out_ptr_t&& rhs) noexcept :
  s(std::move(rhs.s)),
  a(std::move(rhs.a)),
  p(std::move(rhs.p)) {
    rhs.p = nullptr;
}
```

19.11.9.2 Assignment [**out_ptr.class.assign**]

```
out_ptr_t& operator=(out_ptr&& rhs) noexcept;
```

[1] Effects: moves each element of `rhs` into `*this`, then sets `rhs.p` to `nullptr` so that the destructor's effects do not apply.

[2] Equivalent to:

```
out_ptr_t& operator=(out_ptr_t&& rhs) noexcept {
  s = std::move(rhs.s);
  a = std::move(rhs.a);
  p = std::move(rhs.p);
  rhs.p = nullptr;
  return *this;
}
```

19.11.9.3 Destructors [**out_ptr.class.dtor**]

```
~out_ptr_t();
```

[1] Let `SP` be `POINTER_OF(Smart)` (**19.10.2**).

[2] Effects: reset the pointer stored in *s*, if *p* is not null using the `Args` value stored in `*this`, if any.

[3] Equivalent to:

— `if (p != nullptr) { s.reset( static_cast<SP>(p), std::forward<Args>(args)... ); }` if `reset` is a valid member function on `Smart`,

— otherwise `if (p != nullptr) { s = Smart( static_cast<SP>(p), std::forward<Args>(args)... ); };`

19.11.9.4 Conversions [**out_ptr.class.conv**]

```
operator Pointer*() noexcept;
operator void**() noexcept; // if Pointer not void*
```

[1] Constraints: The second conversion shall participate in conversion if `Pointer` is not of type `void*`.

[2] Effects: The first conversion returns a pointer to `p`. The second conversion return `static_cast<void**>(static_cast<void*>(static_cast<Pointer*>(*this)));`

Insert §19.11.10 [**out_ptr**]:

19.11.10 **Function Template** `out_ptr` [**out_ptr**]

[1] *out_ptr* is a function template that produces an object of type *out_ptr_t* (**19.11.9**).

```
namespace std {

  template <class Pointer, class Smart, class... Args>
  auto out_ptr(Smart& s, Args&&... args) noexcept
  -> out_ptr_t<Smart, Pointer, Args...>;

  template <class Smart, class... Args>
  auto out_ptr(Smart& s, Args&&... args) noexcept
  -> decltype(out_ptr<Pointer>(s, std::forward<Args>(args)...));

}
```

[2] Effects: For the second overload, let `Pointer` be `POINTER_OF(Smart)` (**19.10.2**).

[3] Equivalent to: `return out_ptr_t<Smart, Pointer, Args...>(s, std::forward<Args>(args)...);`

Insert §19.11.11 [**inout_ptr.class**]:

19.11.11 **Class Template** `inout_ptr_t` [**inout_ptr.class**]

[1] *inout_ptr_t* is a type used with smart pointers (**19.11**) and types which are designed on the same principles to interoperate easily with functions that use output pointer parameters. [Note — For example, a function of the form `void foo(void**)` — end note].

[2] *inout_ptr_t* may be specialized (**12.6.5**) for user-defined types and shall meet the observable behavior in the rest of this section.

```
namespace std {

  template <class Smart, class Pointer, class... Args>
  struct inout_ptr_t {
    // 19.11.11.1, constructors
    inout_ptr_t(Smart&, Args...) noexcept;
    inout_ptr_t(inout_ptr_t&&) noexcept;
    inout_ptr_t(const inout_ptr_t&) noexcept = delete;

    // 19.11.11.2, assignment
    inout_ptr_t& operator=(inout_ptr_t&&) noexcept;
    inout_ptr_t& operator=(const inout_ptr_t&) noexcept = delete;

    // 19.11.11.3, destructors
    ~inout_ptr_t();

    // 19.11.11.4, conversion operators
    operator Pointer*() noexcept; // if Pointer not void*
    operator void**() noexcept;

  private:
    Smart* s; // exposition only
    tuple<Args> a; // exposition only
    Pointer p; // exposition only
  };

}
```

[2] If `Smart` is a specialization of `shared_ptr` and `sizeof...(Args) == 0`, the program is ill-formed. `Pointer` shall meet the `Cpp17NullablePointer` requirements (**15.5.3.3**).

[3] [*Note:* It is typically a user error to reset a `shared_ptr` without specifying a deleter, as `std::shared_ptr` will replace a custom deleter with the default deleter upon usage of `.reset()`, as specified in 19.11.3.4. — *end Note*]

19.11.11.1 Constructors [**inout_ptr.class.ctor**]

```
inout_ptr_t(Smart& smart, Args... args) noexcept;
```

[1] Constraints: `s.release()` must be a valid expression.

[1] Effects: constructs an object of `inout_ptr_t` and stores the arguments to be used for destructor.

[2] Equivalent to:

```
inout_ptr_t(Smart& smart, Args... args) noexcept :
  s(&smart),
  a(std::forward<Args>(args)...),
  p(static_cast<Pointer>(smart.release()))
{}.
```

```
inout_ptr_t(inout_ptr&& rhs) noexcept;
```

[3] Effects: moves all elements of `rhs` into `*this`, then sets `rhs.p` to `nullptr` so that the destructor's effects do not apply.

[4] Equivalent to:

```
inout_ptr_t(inout_ptr_t&& rhs) noexcept :
  s(std::move(rhs.s)),
  a(std::move(rhs.a)),
  p(std::move(rhs.p)) {
    rhs.p = nullptr;
}
```

19.11.11.2 Assignment [**inout_ptr.class.assign**]

```
inout_ptr_t& operator=(inout_ptr&& rhs) noexcept;
```

[1] Effects: moves each element of `rhs` into `*this`, then sets `rhs.p` to `nullptr` so that the destructor's effects do not apply.

[2] Equivalent to:

```
inout_ptr_t& operator=(inout_ptr_t&& rhs) noexcept {
  s = std::move(rhs.s);
  a = std::move(rhs.a);
  p = std::move(rhs.p);
  rhs.p = nullptr;
  return *this;
}
```

19.11.11.3 Destructors [**inout_ptr.class.dtor**]

```
~inout_ptr_t();
```

[1] Let `SP` be `POINTER_OF(Smart)` (**19.10.2**).

[2] Effects: reset the pointer stored in *s*, if *p* is not null using the `Args` value stored in `*this`, if any.

[3] Equivalent to:

— `if (p != nullptr) { s.reset( static_cast<SP>(p), std::forward<Args>(args)... ); }` if `reset` is a valid member function on `Smart`,

— otherwise `if (p != nullptr) { s = Smart( static_cast<SP>(p), std::forward<Args>(args)... ); };`

19.11.11.4 Conversions [**inout_ptr.class.conv**]

```
operator Pointer*() noexcept;
operator void**() noexcept; // if Pointer not void*
```

[1] Constraints: The second conversion shall participate in conversion if `Pointer` is not of type `void*`.

[2] Effects: The first conversion returns a pointer to `p`. The second conversion return `static_cast<void**>(static_cast<void*>(static_cast<Pointer*>(*this)));`

Insert §19.11.12 [**inout_ptr**]:

> 19.11.12 **Function Template** `inout_ptr` [**inout_ptr**]
>
> [1] *inout_ptr* is a function template that produces an object of type *inout_ptr_t* (**19.11.11**).
>
> ```
> namespace std {
>
>   template <class Pointer, class Smart, class... Args>
>   auto inout_ptr(Smart& s, Args&&... args) noexcept
>   -> inout_ptr_t<Smart, Pointer, Args...>;
>
>   template <class Smart, class... Args>
>   auto inout_ptr(Smart& s, Args&&... args) noexcept
>   -> decltype(inout_ptr<Pointer>(s, std::forward<Args>(args)...));
>
> }
> ```
>
> [2] Effects: For the second overload, let `Pointer` be `POINTER_OF(Smart)` (**19.10.2**).
>
> [3] Equivalent to: `return inout_ptr_t<Smart, Pointer, Args...>(s, std::forward<Args>(args)...);`

§ 8. Acknowledgements

Thank you to Lounge<C++>'s Cicada, melak47, rmf, and Puppy for reporting their initial experiences with such an abstraction nearly 5 years ago and helping JeanHeyd Meneide implement the first version of this.

Thank you to Mark Zeren for help in this investigation and analysis of the performance of smart pointers.

§ References

§ Informative References

[CCOMPTR]

Microsoft. CComPtr::operator& Operator. 2015. URL: https://msdn.microsoft.com/en-us/library/31k6d0k7.aspx

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. N4750 - Working Draft, Standard for Programming Language C++. May 7th, 2018. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[P0468]

Isabella Muerte. A Proposal to Add an Intrusive Smart Pointer to the C++ Standard Library. October 15th, 2016. URL: http://wg21.link/p0468

[WRL-COMPTRREF]

Microsoft. ComPtrRef Class. November 4th, 2016. URL: https://docs.microsoft.com/en-us/cpp/windows/comptrref-class